

Relatório Técnico

FoodDelivery

Angel Yraê Ribeiro Ferreira, Augusto Brandão, Carlos Henrique de Souza Santana Santiago, João Guilherme Gonçalves Pinheiros, Rafael Santos da Silva

Resumo

O projeto consiste na reestruturação do código existente para aplicação de padrões de projeto, com foco no padrão Facade. O sistema, desenvolvido em Java, simula o gerenciamento de pedidos de restaurantes, permitindo o cadastro de clientes, itens de cardápio e criação de pedidos com controle de status. A refatoração visou melhorar a legibilidade, manutenção e extensibilidade do código, mantendo o comportamento original da aplicação. Foram adotadas práticas de Programação Orientada a Objetos e princípios de design limpo para garantir a escalabilidade e a reutilização dos componentes.

1. Introdução

O sistema FoodDelivery é uma aplicação voltada à automação de pedidos em restaurantes, permitindo gerenciar clientes, cardápios e fluxos de pedidos de forma eficiente. No contexto do que foi solicitado, a equipe Cidade do México foi responsável pela implementação do padrão de projeto Facade, pertencente à categoria estrutural, com o objetivo de fornecer uma interface unificada para um conjunto de interfaces de um subsistema complexo, conforme definido pela coordenação do projeto.

A refatoração teve como objetivo principal aprimorar a estrutura do código, tornando-o mais modular e de fácil expansão. O uso de padrões de projeto é essencial para evitar código duplicado, facilitar a manutenção e aumentar a flexibilidade, principalmente em sistemas que simulam múltiplas etapas, como o ciclo de vida de um pedido.

2. Fundamentação Teórica

A Programação Orientada a Objetos (POO) é o paradigma central do desenvolvimento do FoodDelivery. Ela permite representar entidades do mundo real por meio de classes e objetos, promovendo a reutilização e o encapsulamento. Conceitos como herança, abstração e composição foram aplicados para estruturar o sistema.

O padrão de projeto Builder é um padrão criacional cujo propósito é separar a construção

de um objeto complexo da sua representação final. No sistema, a criação de um pedido envolve múltiplas etapas como adicionar cliente, incluir itens, definir quantidades, o que tornava o processo suscetível a erros e difícil de ler. Com o Facade, ao invés do cliente interagir diretamente com múltiplas classes, o padrão Facade introduz uma classe intermediária como SistemaPedidosFacade, que centraliza e simplifica o acesso a essas operações.

3. Metodologia

O repositório foi clonado a partir do repositório principal da FoodDelivery, e o desenvolvimento foi realizado em Java utilizando a linha de comando como interface de interação. A implementação seguiu uma metodologia incremental, com commits documentados e revisões contínuas.

As principais etapas foram:

- Análise do código existente: identificação das dependências diretas entre classes e serviços.
- Criação da classe `SistemaPedidosFacade`: ponto central para operações como `registrarCliente()`, `criarPedido()` e `listarCardapio()`.
- Encapsulamento das interações internas: a fachada passou a interagir com os serviços internos, ocultando a complexidade do subsistema.
- Refatoração da interface CLI: a linha de comando passou a se comunicar apenas com a fachada.
- Testes e code review: execução de casos de uso e revisão cruzada entre equipes para validação da refatoração.

4. Resultados e Discussões

A refatoração resultou em uma melhor organização e separação da lógica de criação de pedidos. O uso do Builder eliminou duplicações e centralizou o processo de construção, garantindo maior clareza na sequência de operações.

Principais resultados obtidos:

- O método de criação de pedidos tornou-se mais seguro e menos propenso a inconsistências.
- A leitura e manutenção do código melhoraram significativamente.
- O sistema está pronto para receber novos recursos, como integração com banco de dados ou interface gráfica.
- Além disso, a aplicação manteve compatibilidade com os outros padrões de projeto descritos na OAT (como State e Observer), demonstrando o potencial de expansão do sistema em futuras etapas.

Desafios enfrentados:

- Adequar o código existente ao padrão sem alterar o comportamento externo da aplicação.
- Manter a coesão entre as classes de cliente, cardápio e pedido.
- Garantir o funcionamento da CLI após a refatoração.
- A equipe solucionou esses desafios com revisões frequentes, testes e análise conjunta do design.

5. Considerações Finais

O desenvolvimento da versão refatorada do FoodDelivery reforçou a importância dos padrões de projeto como ferramentas essenciais na engenharia de software. O padrão Builder* mostrou-se eficaz na separação entre a lógica de montagem e a representação final dos objetos, tornando o sistema mais robusto e modular.

Como trabalhos futuros, sugere-se:

- Implementar persistência em banco de dados;
- Integrar o padrão Observer para notificação de eventos;
- Criar testes automatizados para validar fluxos de uso.

A experiência proporcionou à equipe uma visão mais profunda sobre design orientado a objetos e refatoração limpa, aproximando o projeto de práticas profissionais de mercado.

6. Referências

FOWLER, M. *Refactoring: Improving the Design of Existing Code.* Addison-Wesley, 1999.

GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley, 1995.

HORSTMANN, C. *Core Java Volume I – Fundamentals.* Prentice Hall, 2019.

SOMMERVILLE, I. *Software Engineering.* 9th ed. Addison-Wesley, 2011.

BOOCH, G. *Object-Oriented Analysis and Design with Applications.* Addison-Wesley, 2007.